



Trellix – Technical Feasibility Study

Author: Issa El Mokadem

Program: 2º DAW – I.E.S. Abdera

Project Type: Full-Stack Web Application (Task Board)

1. Introduction

This document presents the Technical Feasibility Study for **Trellix**, a task-management web application inspired by Trello.

Purpose of this study:

- Validate that the chosen technologies are appropriate
 - Ensure the project can be completed within the available time
 - Confirm the solution meets DAW Final Project requirements
 - Identify risks and limitations
-

2. Objectives of the Study

- Ensure selected technologies comply with official project requirements
 - Confirm developer skills match project requirements
 - Identify technical risks early
 - Evaluate deployment complexity (Docker + Nginx)
 - Determine feasibility within available timeframe
-

3. Project Description

Trellix is a Kanban-style task board featuring:

- Boards, lists, and cards

- Drag & drop interactions
- User assignment
- Deadlines and reminders
- User roles (Administrator / User)
- External API integration (e.g., notifications, time-tracking)

 Note: During implementation, scope was adjusted. User assignment, deadlines, reminders, and additional API integrations were deprioritized in favor of polishing core features (boards, lists, cards, drag & drop, auth, weather widget).

4. Technologies

4.1 Frontend

- HTML5
- Tailwind CSS
- React (SPA, state management, reusable components)

4.2 Backend

- Python + FastAPI
- CRUD operations
- Authentication with JWT
- RBAC (Role-Based Access Control)

4.3 Database

- MariaDB

4.4 Deployment

- Docker + docker-compose
- Nginx (reverse proxy)

Containers: Frontend, Backend, Nginx, MariaDB

4.5 Local Domain

- `elmokadem-issa.proyecto-daw.iesabdera.local`
- Optional cloud deployment via Render or other platforms

4.6 Git & GitHub

- Feature-based branching

- Frequent commits
- Complete README documentation
- Integration with Jira for Agile workflow

4.7 External API

- Open Meteo
 - Notification services
-

5. Technical Requirements

5.1 Functional Requirements

- Full CRUD operations
- User & role management
- Modern React UI
- Drag & drop interactions
- Notification system
- External API integration

5.2 Non-Functional Requirements

- Secure authentication using JWT
 - Good performance and response times
 - Decoupled architecture
 - Responsive design
 - Clean and well-documented code
-

6. Skills Assessment

Area	Level	Notes
HTML/CSS	High	No difficulty expected
Tailwind CSS	Learning	Rapid acquisition expected
React	Intermediate	Learning but solid foundation

FastAPI	Medium-High	Prior experience with RBAC & auth
MariaDB	Medium-High	Previously used in smaller projects
Docker	Basic	Needs learning
Nginx	Basic	Expected to learn quickly
RBAC + JWT	Medium	Requires minor refresh

✔ Despite ongoing learning in React, Tailwind, and Nginx, the technology stack is manageable and appropriate for the project scope.

7. Required Resources

- VSCode
- Python 3.12
- Docker
- Git
- Web bundlers

ℹ All tools are free and accessible.

8. Feasibility Analysis

Technology	Viable	Reason
React + Tailwind	✔ Yes	Ideal for dynamic UI
FastAPI	✔ Yes	Lightweight, fast, familiar
MariaDB	✔ Yes	Stable, known database
Docker	✔ Yes	Easy service isolation
Nginx Proxy	✔ Yes	Standard configuration

External API	✅ Yes	Easy integration
RBAC	✅ Yes	Native to FastAPI

- ✅ The project is fully viable given the chosen stack and available timeframe.

9. Identified Risks

Risk	Probability	Mitigation
Complex drag & drop	Medium-High	Use libraries like dnd-kit
Nginx configuration	Medium-Low	Follow official documentation
Docker-compose errors	Medium-High	Test services individually first
JWT security issues	Low	Follow recommended patterns
Feature creep	High	Strict MVP definition

- ⚠️ Risks should be monitored closely during development.

10. Final Conclusion

- ✅ Trellix is technically feasible and realistic.
 - All necessary tools and skills are available or achievable
 - Project aligns with the DAW Final Project rubric
 - Demonstrates full-stack development skills
 - Architecture, requirements, and risks are well-defined, making execution manageable